NESNE TEMELLİ PROGRAMLAMA PRENSİPLERİ

Araştırma Çalışması

Bilgi	Açıklama
Adı Soyadı	Berke HARMAN
Öğrenci No	256001044
Konu	Nesne Temelli Programlama (OOP) – Temel Prensipler
Örnek Dil	C# (.NET)

1. Giriş: Nesne Temelli Programlama Nedir?

Nesne Temelli Programlama (Object-Oriented Programming – OOP), bir yazılımı birbirinden bağımsız komutlar ve fonksiyonlar yığını olarak değil, **kendi verisi ve davranışı olan nesnelerin** bir araya gelmesi olarak tasarlama yaklaşımıdır. Gerçek dünyada da her şeyi nesneler üzerinden düşünürüz: bir arabanın rengi, markası ve hızı (veri) vardır; aynı zamanda hızlanır, fren yapar ve korna çalar (davranış). OOP, yazılımdaki kavramları da bu şekilde modellememizi sağlar.

Prosedürel programlamada veriler ve bu verileri işleyen fonksiyonlar ayrı yerlerde durur. Program büyüdükçe hangi fonksiyonun hangi veriyi değiştirdiğini takip etmek zorlaşır, küçük bir değişiklik programın birçok yerini etkiler. OOP ise veriyi ve o veriyle ilgili işlemleri aynı yapı (sınıf) içinde toplar. Böylece kod daha **düzenli, tekrar kullanılabilir, bakımı kolay ve genişletilebilir** hâle gelir.

Bu çalışmada OOP'nin temel kavramlarını; her birinin **ne olduğunu, hangi problemi çözdüğünü, günlük hayattaki karşılığını** ve küçük C# kod örnekleriyle **gerçek bir projede nasıl kullanılabileceğini** açıklamaya çalıştım. Gerçek proje örneği olarak dönem projem olan **Akıllı Yurt Asistanı** uygulamasını kullandım.

2. Sınıf (Class) ve Nesne (Object)

Sınıf,bir nesnenin hangi özelliklere(alan/property) ve hangi davranışlara (metot) sahip olacağını tanımlayan bir şablondur. **Nesne** ise bu şablondan üretilmiş, bellekte gerçekten yer kaplayan somut bir örnektir (instance). Bir sınıftan istediğimiz kadar nesne oluşturabiliriz ve her nesnenin değerleri birbirinden bağımsızdır.

Günlük hayattan örnek

Bir mimarın çizdiği **ev projesi** sınıftır; o projeye bakılarak inşa edilen her bir **ev** ise nesnedir. Aynı projeden yapılan evlerin oda sayısı aynıdır, ama birinin duvarı mavi diğerininki beyaz olabilir. Benzer şekilde "Öğrenci" bir sınıftır; "Ahmet" ve "Ayşe" bu sınıftan oluşturulmuş iki farklı nesnedir.

Hangi problemi çözer?

Aynı yapıdaki verileri (örneğin 500 öğrencinin adı, numarası, oda bilgisi) ayrı ayrı değişkenlerde tutmak yerine tek bir tanımla düzenli şekilde modellememizi sağlar. Veri ve o veriyle ilgili işlemler bir arada durduğu için kod okunabilir olur.

Kod örneği

```
public class Ogrenci
{
      public string Ad { get; set; } public
      string OdaNo { get; set; }

      public void Tanit()
      {
}
            Console.WriteLine($"Merhaba, ben {Ad}. {OdaNo} numaralı odada kalıyorum.");
      }
// Sınıftan nesne oluşturma
Ogrenci ogr1 = new Ogrenci { Ad = "Ahmet", OdaNo = "B-204" };
Ogrenci ogr2 = new Ogrenci { Ad = "Ayşe", OdaNo = "A-112" };



ogr1.Tanit();          // Merhaba, ben Ahmet. B-204 numaralı odada kalıyorum.
```

Gerçek projede kullanımı

Akıllı Yurt Asistanı'nda Ogrenci, Oda, ArizaTalebi, Duyuru, Sensor gibi her kavram bir sınıf olarak tanımlanır. Veritabanındaki her satır, Entity Framework tarafından bu sınıflardan bir nesneye dönüştürülür.

3. Constructor (Yapıcı Metot) ve Nesne Oluşturma Mantığı

Constructor, bir nesne new anahtar kelimesiyle oluşturulduğu anda **otomatik olarak çalışan** özel bir metottur. Adı sınıf adıyla aynıdır ve geri dönüş tipi yoktur. Görevi, nesneyi kullanılmaya hazır ve **geçerli bir başlangıç durumuna** getirmektir. Bir sınıfta birden fazla constructor tanımlanabilir (overloading); hiç yazılmazsa derleyici parametresiz bir varsayılan constructor ekler.

new Oda("B-204", 3) yazdığımızda arka planda sırasıyla şunlar olur: (1) bellekte (heap) nesne için yer ayrılır, (2) alanlar varsayılan değerlerini alır (sayılar 0, referanslar null), (3) constructor çalışarak alanlara asıl değerlerini verir, (4) oluşan nesnenin adresi (referansı) değişkene atanır.

Günlük hayattan örnek

Yeni bir telefon kutusundan çıkıp ilk açıldığında "kurulum sihirbazı" çalışır: dil, saat ve Wi-Fi ayarları yapılmadan telefon kullanılamaz. Constructor da nesnenin bu ilk kurulumudur.

Hangi problemi çözer?

Nesnenin eksik ya da hatalı değerlerle oluşturulmasını engeller. Örneğin kapasitesi negatif olan bir oda veya adı boş olan bir öğrenci nesnesi hiç oluşturulamaz.

Kod örneği

```
public class Oda
{
	public string OdaNo { get; }
	public int Kapasite { get; }

	// Parametreli constructor
	public Oda(string odaNo, int kapasite)
	{
		if (kapasite <= 0)
			throw new ArgumentException("Kapasite 0'dan büyük olmalıdır.");
		OdaNo = odaNo;
		Kapasite = kapasite;
}	}

	// Overload: kapasite verilmezse varsayılan 2 kişilik oda
	public Oda(string odaNo) : this(odaNo, 2) { }
var oda1 = new Oda("B-204", 3); var
oda2 = new Oda("A-101");



                                                    // Kapasite = 2
```

4. Erişim Belirleyiciler (Access Modifiers)

Erişim belirleyiciler, bir sınıfın veya sınıf üyesinin (alan, property, metot) **kimler tarafından görülebileceğini ve kullanılabileceğini** belirler. Kapsüllemenin uygulanmasını sağlayan araçlardır. C#'ta en sık kullanılanlar şunlardır:

Belirleyici	Erişim Alanı	Günlük HayatKarşılığı
public	Her yerden erişilebilir.	Yurdun giriş holündeki duyuru panosu – herkes görebilir.
private	Sadece tanımlandığı sınıfın içinden erişilebilir.	Kişisel dolabın – sadece sen açabilirsin.
protected	Tanımlandığı sınıf ve ondan türeyen (alt) sınıflar erişebilir.	Aile fotoğraf albümü – sadece aile bireyleri bakabilir.
internal	Aynı proje (assembly) içinden erişilebilir.	Sadece personelin girebildiği yurt ofisi.

C#'ta sınıf üyelerinin varsayılan erişimi private'tır. Genel kural; bir üyeyi **olabildiğince kısıtlı** tanımlamak ve yalnızca gerçekten dışarıya açılması gerekenleri public yapmaktır.

Kod örneği

```
public class Kullanici
{
    public string Ad { get; set; } private string sifreHash;
    protected DateTime kayitTarihi;

    public void SifreBelirle(string sifre)
    {
        sifreHash = Hashle(sifre);
    }
    private string Hashle(string s)
    {
        byte[] veri = Encoding.UTF8.GetBytes(s);
        return Convert.ToBase64String(SHA256.HashData(veri));
    }
}
var k = new Kullanici();
k.Ad = "Furkan";
// k.sifreHash = "123";
```

```
// herkes erişebilir
// sadece bu sınıf erişebilir
// bu sınıf ve alt sınıflar erişebilir

// private metoda sınıf içinden erişim

// geçerli
// HATA: private alana dışarıdan erişilemez
```

5. Kapsülleme (Encapsulation)

Kapsülleme, bir nesnenin verisini dış dünyadan gizleyip bu veriye yalnızca nesnenin kendi belirlediği kontrollü yollarla (metotlar ve property'ler) erişilmesini sağlamaktır. Alanlar private yapılır; okuma/yazma işlemleri doğrulama içeren public metotlar veya property'ler üzerinden yapılır.

Günlük hayattan örnek

Bir **ATM**'de bankanın kasasına doğrudan elimizi sokup para alamayız. Sadece ekrandaki menüyü (arayüz) kullanırız; ATM kartımızı, şifremizi ve bakiyemizi kontrol ettikten sonra işlemi yapar. Kasadaki para (veri) gizlenmiş, erişim kurallara bağlanmıştır.

Hangi problemi çözer?

Verinin dışarıdan kontrolsüz şekilde değiştirilip **geçersiz bir duruma düşmesini** engeller (örneğin bakiyenin eksiye düşmesi). Ayrıca sınıfın iç yapısı değişse bile dışarıdaki kodu etkilemez; çünkü dışarısı sadece public arayüzü kullanır.

Kod örneği

```
public class CamasirMakinesi
{
    public string MakineNo { get; }
    public int KalanDakika { get; private set; }                    // dışarıdan sadece okunabilir
    public bool Dolu => KalanDakika > 0;

    public CamasirMakinesi(string no) => MakineNo = no;
    public void Baslat(int dakika)
    {

        if (Dolu)
            throw new InvalidOperationException("Makine şu anda kullanımda.");
}       if (dakika < 15 || dakika > 120)
            throw new ArgumentException("Süre 15-120 dakika arasında olmalıdır.");
var m = new CamasirMakinesi("M-3");      KalanDakika = dakika;
m.Baslat(60);                       }
// m.KalanDakika = -10; // HATA:setprivate,değerdışarıdanbozulamaz
```

Gerçek projede kullanımı

Projede bir arıza talebinin durumu (Beklemede → Atandı → Çözüldü) doğrudan değiştirilemez; TalebiAta() ve Coz() gibi metotlarla, sıra kurallarına uyularak değiştirilir. Böylece "çözülmüş bir talebin tekrar beklemeye alınması" gibi hatalar engellenir.

6. Soyutlama (Abstraction)

Soyutlama, bir nesnenin karmaşık iç detaylarını gizleyip kullanıcıya sadece **ne yaptığını** gösteren basit bir arayüz sunmaktır. Kapsülleme "veriyi korumak" ile ilgiliyken soyutlama "karmaşıklığı gizleyip önemli olana odaklanmak" ile ilgilidir. C#'ta soyutlama genellikle interface ve abstract class ile yapılır.

Günlük hayattan örnek

Araba kullanırken sadece direksiyon, gaz ve fren pedalını kullanırız. Motorun içinde yakıtın nasıl yandığını, şanzımanın nasıl vites değiştirdiğini bilmemize gerek yoktur. Televizyon kumandası da öyledir: "kanal değiştir" tuşuna basarız, içerideki elektronik devre bizden gizlenmiştir.

Hangi problemi çözer?

Programın farklı parçalarının birbirinin iç detayına bağımlı olmasını engeller. Kodu kullanan kişi sadece "ne yapılacağını" bilir, "nasıl yapılacağı" değişse bile kodu etkilenmez. Bu da büyük projelerde karmaşıklığı yönetilebilir hâle getirir.

Kod örneği

```
// Ne yapılacağı tanımlanır, nasıl yapılacağı gizlenir public
interface IBildirimServisi
{
      void Gonder(string alici, string mesaj);
}

public class PushBildirimServisi : IBildirimServisi
{
      public void Gonder(string alici, string mesaj)
      {
            // Firebase bağlantısı, token kontrolü, yeniden deneme... (gizli detaylar)
            Console.WriteLine($"[Push] {alici}: {mesaj}");
}
      }
public class ArizaYonetimi
{


      private readonly IBildirimServisi _bildirim;
      public ArizaYonetimi(IBildirimServisi bildirim) => _bildirim = bildirim;

      public void TalepCozuldu(string ogrenci) =>
            _bildirim.Gonder(ogrenci, "Arıza talebiniz çözüldü.");                  // nasıl gittiği
bilinmez
}
```

ArizaYonetimi sınıfı bildirimin push, e-posta ya da SMS ile gittiğini bilmez; sadece “gönder” der. İleride SMS’e geçmek istersek yalnızca yeni bir SmsBildirimServisi sınıfı yazmamız yeterli olur.

7. Kalıtım (Inheritance)

Kalıtım, bir sınıfın (alt sınıf / derived class) başka bir sınıfın (üst sınıf / base class) özelliklerini ve ~~metotlarını devralmasıdır. Alt sınıf, üst sınıftaki her şeye sahip olur ve üzerine kendi özel özellikleri~~ni ekleyebilir veya bazı davranışları değiştirebilir (override). Kalıtım bir **“-dır / is-a” ilişkisi** kurar: “Öğrenci bir Kullanıcıdır.” C#’ta bir sınıf yalnızca **tek bir sınıftan** kalıtım alabilir.

Günlük hayattan örnek

Çocuklar göz rengi, saç rengi gibi özellikleri anne-babalarından alır ama kendilerine has özellikleri de vardır. Benzer şekilde “Taşıt” genel bir kavramdır; “Otomobil” ve “Motosiklet” taşıtın tüm özelliklerini (tekerlek, hız, hareket et) alır, kendilerine özgü özellikler ekler.

Hangi problemi çözer?

Kod tekrarını önler. Öğrenci, yönetici ve teknik personelin hepsinde ad, e-posta, şifre ve giriş yapma işlemi ortaktır. Bunları her sınıfa ayrı ayrı yazmak yerine bir kez Kullanici sınıfına yazarız. Ortak bir hata düzeltildiğinde tüm alt sınıflar otomatik olarak düzelir.

Kod örneği

```
publicclass Kullanici
{
      public string AdSoyad { get; set; }
      public string Eposta { get; set; }
}     public bool GirisYap(string sifre) { /* ortak giriş kontrolü */ return true; }

public class Ogrenci : Kullanici
{
                                                    // Öğrenci bir Kullanıcıdır

}
      public string OdaNo { get; set; }
      public void ArizaBildir(string aciklama) { /* ... */ }
public class Yonetici : Kullanici
{

                                                    // Yönetici bir Kullanıcıdır
}

      public void DuyuruYayinla(string metin) { /* ... */ }
var ogr = new Ogrenci { AdSoyad = "Ahmet Yılmaz", OdaNo = "B-204" };
// Kullanici sınıfından miras alınan metot


ogr.GirisYap("1234");
ogr.ArizaBildir("Musluk akıtıyor");
```

Dikkat: Kalıtım sadece gerçekten “is-a” ilişkisi varsa kullanılmalıdır. Sırf bir metodu yeniden kullanmak için kalıtım yapmak, sınıflar arasında gereksiz ve kırılgan bağımlılıklar oluşturur (bkz. Bölüm 10).

8. Çok Biçimlilik (Polymorphism)

Çok biçimlilik, aynı isimdeki bir metodun farklı nesnelerde **farklı şekillerde davranabilmesidir**. Üst sınıf türünde bir değişken, alt sınıfların herhangi birinin nesnesini tutabilir ve metot çağrıldığında nesnenin gerçek tipine ait metot çalışır. İki türü vardır:

- **Çalışma zamanı çok biçimliliği (overriding):** Üst sınıftaki virtual veya abstract metot, alt sınıfta override ile yeniden yazılır. Hangi metodun çalışacağına program çalışırken karar verilir.
- **Derleme zamanı çok biçimliliği (overloading):** Aynı sınıfta aynı isimli ama farklı parametreli metotlar tanımlanır (örneğin Topla(int, int) ve Topla(double, double)).

Günlük hayattan örnek

Bir öğretmen sınıfa “**Ses çıkarın!**” dese; köpek maskesi takan öğrenci havlar, kedi maskesi takan miyavlar. Komut aynıdır ama herkes kendi “tipine” göre tepki verir. Ya da telefonumuzdaki “Paylaş” tuşu: aynı tuş WhatsApp’ta mesaj, e-postada ek, Instagram’da gönderi olarak farklı çalışır.

Hangi problemi çözer?

Kodda uzun if-else veya switch bloklarıyla “bu nesnenin tipi ne?” diye kontrol etme ihtiyacını ortadan kaldırır. Yeni bir tür eklendiğinde mevcut kodu değiştirmeden sadece yeni bir sınıf eklemek yeterli olur (açık-kapalı prensibi).

Kod örneği

```
publicabstract class Sensor
{
      public string OdaNo { get; set; }
      public abstract double Olc();                                   // her sensör farklı ölçer
}     public virtual string Birim => "";

public class SicaklikSensoru : Sensor
{

}
      public override double Olc() => 23.5;                           // DHT22'den okunur
      public override string Birim => "°C";
public class GazSensoru : Sensor
{

}

      public override double Olc() => 180;                            // MQ-2'den okunur
List<Sensor> sensorler = new() { new SicaklikSensoru(), new GazSensoru() }; foreach
(Sensor s in sensorler)
      public override string Birim => "ppm";




      Console.WriteLine($"{s.Olc()} {s.Birim}");                      // her nesne kendi Olc()
metodunu çalıştırır
```

Döngü, listedeki sensörün sıcaklık mı gaz mı olduğunu bilmeden çalışır. Projeye ileride bir “Nem Sensörü” eklendiğinde bu döngüde hiçbir değişiklik yapmamız gerekmez.

9. Interface ve Abstract Class Kavramları

9.1 Abstract Class (Soyut Sınıf)

abstract anahtar kelimesiyle tanımlanan, **doğrudan nesnesi oluşturulamayan** ve sadece kalıtım için kullanılan sınıftır. İçinde hem gövdesi yazılmış normal metotlar ve alanlar, hem de gövdesi olmayan (alt sınıfların doldurmak **zorunda** olduğu) abstract metotlar bulunabilir. Birbirine çok benzeyen, ortak kodu ve ortak verisi olan sınıflar için ortak bir temel oluşturur.

Günlük hayat: “Hayvan” soyut bir kavramdır; sokakta “sadece hayvan” göremeyiz, ya kedi ya köpek görürüz. Ama tüm hayvanların ortak özellikleri (yaş, nefes alma) vardır.

9.2 Interface (Arayüz)

Interface, bir sınıfın **neler yapabilmesi gerektiğini** belirten bir sözleşmedir. Genellikle sadece metot ve property imzalarını içerir; nasıl yapılacağını, bu interface’i uygulayan (implement eden) sınıf belirler. Bir sınıf yalnızca bir sınıftan kalıtım alabilir ama **birden fazla interface’i** uygulayabilir. Interface’ler bir **“yapabilir / can-do”** ilişkisi kurar.

Günlük hayat: Duvardaki **elektrik prizi** bir interface’tir. Priz, “iki pimli fiş takılabilir” kuralını koyar. Telefon şarj aleti, ütü veya bilgisayar bu kurala uyduğu sürece prize takılabilir; priz hangi cihazın takıldığıyla ilgilenmez.

Kod örneği

```
public interface IRaporlanabilir { string RaporOlustur(); } public interface
IAlarmVerebilir { bool EsikAsildiMi(); }

public abstract class Cihaz                                    // ortak veri + ortak kod
{
    public string MacAdresi { get; set; }
    public DateTime SonBaglanti { get; protected set; }
    public void BaglantiGuncelle() => SonBaglanti = DateTime.Now;
}
    public abstract void VeriGonder();                          // her cihaz kendi yapar

// Bir sınıf + birden fazla interface
public class GazCihazi : Cihaz, IRaporlanabilir, IAlarmVerebilir {



}   public double SonDeger { get; set; }
    public override void VeriGonder() { /* API'ye POST */ BaglantiGuncelle(); }
// var c = new Cihaz();
    public bool EsikAsildiMi() => SonDeger > 300;
    public string RaporOlustur() => $"{MacAdresi} - Son gaz değeri: {SonDeger} ppm";



                                    // HATA: abstract sınıftan nesne oluşturulamaz
```

9.3 Karşılaştırma

Özellik	Abstract Class	Interface
Nesne oluşturma	Oluşturulamaz	Oluşturulamaz
İçerik	Alan, constructor, gövdeli ve gövdesiz metotlar	Metot/property imzaları (sözleşme)
Çoklu kullanım	Bir sınıf sadece 1 abstract sınıftan türeyebilir	Bir sınıf birden çok interface uygulayabilir
Erişim belirleyici	Üyeleri private/protected olabilir	Üyeleri varsayılan olarak public'tir
İlişki türü	"is-a" (Gaz cihazı bir Cihazdır)	"can-do" (Raporlanabilir, Alarm verebilir)
Ne zaman?	Yakın akraba sınıfların ortak kodu ve verisi varsa	Birbiriyle ilgisiz sınıflara ortak bir yetenek kazandırılacaksa

10. Composition ve Inheritance Arasındaki Fark

Inheritance (kalıtım) bir sınıfın başka bir sınıfın türü olduğunu söyler: "**is-a**" (Öğrenci **bir** Kullanıcıdır). **Composition (bileşim)** ise bir sınıfın başka sınıfların nesnelerini **parça olarak içinde barındırmasıdır**: "**has-a**" (Araba bir Motora **sahiptir**). Composition'da işlevsellik miras alınmaz; içerideki nesnelere iş devredilerek (delegation) elde edilir.

Günlük hayattan örnek

Bir **bilgisayar** işlemci, RAM, ekran kartı ve diskten oluşur. Bilgisayar "bir işlemci" değildir; işlemciye **sahiptir**. Ekran kartı bozulduğunda bilgisayarı atmayız, sadece ekran kartını değiştiririz. Composition'ın gücü buradan gelir: parçalar kolayca değiştirilebilir.

Kod örneği – kötü (kalıtım) ve iyi (composition) tasarım

```
// KÖTÜ: Oda, sadece sensör özelliklerini kullanmak için Sensör'den türetilmiş. // "Oda bir sensördür" cümlesi mantıksızdır; ikinci bir sensör de eklenemez. public class Oda : SicaklikSensoru { }

// İYİ: Oda sensörlere SAHİPTİR (has-a)
public class Oda
{
    public string OdaNo { get; }
    private readonly List<Sensor> _sensorler = new();

    public Oda(string odaNo) => OdaNo = odaNo;
    public void SensorEkle(Sensor s) => _sensorler.Add(s);

    public void DurumYazdir()
    {
        foreach (var s in _sensorler)                    // işi parçalara devreder
            Console.WriteLine($"{OdaNo}: {s.Olc()} {s.Birim}");
    }
}
var oda = new Oda("B-204");
oda.SensorEkle(new SicaklikSensoru());
oda.SensorEkle(new GazSensoru());



                                                         // çalışma anında parça eklenebilir
```

Kriter	Inheritance (Kalıtım)	Composition (Bileşim)
İlişki	is-a (bir türüdür)	has-a (sahiptir / içerir)
Bağımlılık	Sıkı bağlı: üst sınıf değişirse tüm alt sınıflar etkilenir	Gevşek bağlı: parçalar bağımsız değiştirilebilir
Esneklik	İlişki derleme zamanında sabittir	Parçalar çalışma anında değiştirilebilir/eklenebilir
Kod tekrar kullanımı	Üst sınıftan miras alarak	İçteki nesnelere iş devrederek
Risk	Derin ve karmaşık kalıtım ağaçları oluşabilir	Daha fazla küçük sınıf yazılması gerekebilir

Gang of Four olarak bilinen yazarların “Design Patterns” kitabındaki meşhur tavsiye: “**Sınıf kalıtımı yerine nesne bileşimini tercih edin**” (favor object composition over class inheritance). Bu, kalıtımın kötü olduğu anlamına gelmez; gerçek bir “is-a” ilişkisi varsa kalıtım, sadece kod paylaşımı amaçlanıyorsa composition tercih edilmelidir.

11. Nesneler Arasındaki İlişkiler

Gerçek bir yazılımda nesneler tek başına çalışmaz; birbirleriyle iletişim kurar ve birbirlerini kullanırlar. UML’de bu ilişkiler, güçlüden zayıfa doğru şu şekilde sınıflandırılır:

İlişki	Açıklama	Günlük Hayat	AkıllıYurt Projesi
Kalıtım (Generalization)	“is-a” ilişkisi. Alt sınıf üst sınıfın bir türüdür.	Kedi bir Hayvandır.	Ogrenci → Kullanici
Composition (Bileşim)	Güçlü “has-a”. Parça, bütün olmadan var olamaz; bütün silinince parçalar da silinir.	Ev ve odaları: ev yıkılırsa odaları da yok olur.	Bina → Oda
Aggregation (Kümelenme)	Zayıf “has-a”. Parça bütünden bağımsız yaşayabilir.	Takım ve oyuncular: takım dağılsa da oyuncular var olur.	Oda → Ogrenci (öğrenci odadan çıksa da sistemde kalır)

İlişki	Açıklama	Günlük Hayat	Akıllı Yurt Projesi
Association (Birliktelik)	İki nesnenin birbirini tanıdığı, sahiplik içermeyen ilişki.	Doktor ve hasta.	Teknik personel ↔ Arıza talebi
Dependency (Bağımlılık)	En zayıf ilişki. Bir sınıf diğerini sadece bir metodun içinde geçici olarak kullanır.	Yemek yaparken tarifi kullanmak.	RaporServisi metodu parametre olarak Tarih aralığı alır

Kod örneği

```
// COMPOSITION: Odalar Bina içinde oluşturulur, bina yoksa oda da yoktur
public class Bina
{
	public string Ad { get; }
	private readonly List<Oda> _odalar = new();
	public Bina(string ad, int odaSayisi)
	{
		Ad = ad;
		for (int i = 1; i <= odaSayisi; i++)
			_odalar.Add(new Oda($"{ad}-{i:000}"));		// parçalar bütünle doğar
	}
}

// AGGREGATION: Öğrenci dışarıda oluşur, odaya eklenir; oda silinse de öğrenci yaşar public class YurtOdasi
{

}
	public List<Ogrenci> Sakinler { get; } = new();
	public void Yerlestir(Ogrenci o) => Sakinler.Add(o);

// ASSOCIATION: Talep, oluşturan personel tanımlanana sahip değildir
public class ArizaTalebi
{

}

// DEPENDENCY: Rapor servisi, IBildirimServisi'ni yalnızca bu metotta kullanır public class RaporServisi
{
	public TeknikPersonel AtananPersonel { get; set; }

}



	public void HaftalikRaporGonder(IBildirimServisi servis) =>
		servis.Gonder("yonetici", "Haftalık rapor hazır.");
```

12. Kavramların Gerçek Bir Projede Birlikte Kullanımı

Aşağıdaki tabloda, araştırdığım kavramların Akıllı Yurt Asistanı projesinde nerede ve neden kullanıldığı özetlenmiştir. Bu tablo, kavramların tek başına değil **birlikte** çalışarak bir yazılımı düzenli ve genişletilebilir hâle getirdiğini göstermektedir.

Kavram	Projedeki Kullanımı	Sağladığı Fayda
Sınıf / Nesne	Ogrenci, Oda, ArizaTalebi, Sensor sınıfları; veritabanı satırları nesnelere dönüşür.	Veri ve davranış bir arada, okunabilir kod.
Constructor	Oda ve Rezervasyon nesneleri geçerli değerlerle oluşturulur.	Hatalı nesne oluşmaz.
Access Modifiers	Şifre hash'i private, ortak bilgiler public, kayıt tarihi protected.	Hassas veriler korunur.
Kapsülleme	Talep durumu sadece TalebiAta() ve Coz() metotlarıyla değişir.	İş kuralları ihlal edilemez.

Kavram	Projedeki Kullanımı	Sağladığı Fayda
Soyutlama	IBildirimServisi ile bildirimin nasıl gönderildiği gizlenir.	Push yerine SMS'e geçmek kolaydır.
Kalıtım	Ogrenci, Yonetici, TeknikPersonel → Kullanici.	Ortak giriş/profil kodu tek yerde.
Çok biçimlilik	Tüm sensörler tek listede gezilip Olc() çağrılır.	Yeni sensör eklemek mevcut kodu bozmaz.
Interface / Abstract	Cihaz soyut sınıfı; IRaporlanabilir, IAlarmVerebilir arayüzleri.	Ortak kod + esnek yetenekler.
Composition	Oda, sensör listesine sahiptir; Bina, odalardan oluşur.	Parçalar bağımsız değiştirilebilir.

13. Sonuç

Bu araştırmasonucunda OOP'nin bir grup kural ezberinden çok, **gerçek dünyayı yazılımda düzenli bir şekilde modelleme yöntemi** olduğunu anladım. Sınıf ve nesneler gerçek dünyadaki varlıkları temsil ederken; kapsülleme veriyi korur, soyutlama karmaşıklığı gizler, kalıtım kod tekrarını azaltır, çok biçimlilik ise yeni özellik eklemeyi mevcut kodu bozmadan mümkün kılar. Interface ve abstract class'lar sınıflar arasında sözleşmeler kurar; composition ise kalıtıma göre daha esnek ve değiştirilebilir tasarımlar sağlar. En önemli çıkarımım, bu kavramların doğru yerde kullanılması gerektiğidir. Örneğin her ortak kodu kalıtımla çözmeye çalışmak yerine önce "bu gerçekten bir is-a ilişkisi mi?" diye düşünmek gerekir. Bu prensipleri kendi projem olan Akıllı Yurt Asistanı'nda uygulayarak hem daha az hata içeren hem de ileride kolayca geliştirilebilecek bir yazılım oluşturmayı hedefliyorum.

Kaynakça

1. Booch,G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J. ve Houston, K. A. (2007). Object-oriented analysis and design with applications (3. baskı). Boston: Addison-Wesley.
2. Gamma, E., Helm, R., Johnson, R. ve Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Reading, MA: Addison-Wesley.
3. Microsoft. (t.y.). Access modifiers (C# reference). Erişim adresi: https://learn.microsoft.com/dotnet/csharp/language-reference/keywords/access-modifiers (Erişim tarihi: 28.10.2026)
4. Microsoft. (t.y.). Interfaces – define behavior for multiple types. Erişim adresi: https://learn.microsoft.com/dotnet/csharp/fundamentals/types/interfaces (Erişim tarihi: 28.10.2026)
5. Microsoft. (t.y.). Object-oriented programming (C#). Erişim adresi: https://learn.microsoft.com/dotnet/csharp/fundamentals/tutorials/oop (Erişim tarihi: 28.10.2026)
6. Microsoft. (t.y.). Overview of object oriented techniques in C#. Erişim adresi: https://learn.microsoft.com/dotnet/csharp/fundamentals/object-oriented/ (Erişim tarihi: 28.10.2026)